

Day 5 复习总表：JavaScript 基础入门（语法篇）

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2 创建并引入 script.js	新建 JS 文件并接入页面	<pre>html<script src="script.js"></script> (放在 </body> 前)jsconsole.log("Hello, JavaScript!");</pre>	• <code><script></code> 必须成对出现，和自闭合的 <code><link></code> 不同 • 放在 <code></body></code> 之前，确保 HTML 先加载完，JS 再执行，避免找不到页面元素 • <code>console.log()</code> 把内容打印到浏览器控制台，是最常用的调试手段
Step 3 变量与数据类型	声明并打印不同类型的变量	<pre>jslet userName = "Frank";let userAge = 25;let isLearning = true;const birthYear = 1999;</pre>	• <code>let</code> 声明可变变量，<code>const</code> 声明不可变变量 • 字符串用引号包裹；数字不加引号；布尔值只有 <code>true</code>/<code>false</code> • <code>//</code> 是单行注释
Step 4 运算符与表达式	算术运算、字符串拼接、比较运算	<pre>jsconsole.log(a + b, a % b);let fullName = firstName + " " + lastName;console.log(a === b);</pre>	• <code>+</code> <code>-</code> <code>*</code> <code>/</code> 算术运算符，<code>%</code> 取余（模运算） • 字符串用 <code>+</code> 拼接 • <code>===</code> 严格相等（类型和值都要相同），要和赋值符号 <code>=</code> 区分开
Step 5 条件判断 if-else	根据条件执行不同逻辑	<pre>jsif (age >= 18) { ... }else { ... }let isAdult = age >= 18 ? "Yes" : "No";</pre>	• <code>if (条件) { ... } else { ... }</code>：条件为真/假分别执行不同代码块 • <code>else if</code> 可以串联多个条件分支 • 三元运算符 <code>条件 ? 真值 : 假值</code> 是 if-else 的单行简写
Step 6 循环 for/while	重复执行代码，计算累加和	<pre>jsfor (let i = 1; i <= 5; i++) { ... }let sum = 0;for (let i = 1; i <= 100; i++) { sum += i; }</pre>	• <code>for (初始化; 条件; 自增)</code> 适合已知循环次数的场景 • <code>while (条件)</code> 适合不确定循环次数、只知道结束条件的场景 • 累加模式：<code>sum = sum + i</code> 每次循环把当前值加进总和里
Step 7 综合练习	问候程序 + 1到100偶数和	<pre>jslet currentHour = new Date().getHours();if (currentHour < 12) {</pre>	• <code>new Date().getHours()</code> 获取当前小时数 (0-23) • 结合 if-else 根据时间段输出不同问候语

Step 项	完成事项	对应代码 / 指令	代码详解
		<pre>... } if (i % 2 === 0) { evenSum += i; }</pre>	• <code>i % 2 === 0</code> 是判断偶数的标准写法
Step 8 Git 提交	保存本日 JS 基础代 码	<pre>bash git add index.html script.js git commit -m "feat: add JavaScript basics..."</pre>	• 一次提交打包"引入 JS + 变量/运算符/条件/循环"整套基础语法练习
Step 9 Code Review + 自评	通过问答检 验语法理解	考察问题： 1. <code>let</code> 和 <code>const</code> 的区别 2. <code>=</code>、<code>==</code>、<code>===</code> 三者的区别 3. <code>for</code> 和 <code>while</code> 分别适合什么场景	• <code>let</code> 可重新赋值，<code>const</code> 一旦赋值不可更改 • <code>=</code> 赋值；<code>==</code> 比较值（会做类型转换）；<code>===</code> 严格相等（类型和值都要求一致） • <code>for</code> 适合已知循环次数；<code>while</code> 适合按条件判断、次数不确定的场景

本日核心概念速查

概念	一句话记忆
<code>let</code> vs <code>const</code>	会变用 <code>let</code> ，不变用 <code>const</code> ，默认优先用 <code>const</code>
<code>===</code> vs <code>==</code>	写代码时永远优先用 <code>===</code> ，避免隐式类型转换的坑
三元运算符	简单的 if-else 二选一，可以压缩成一行
<code>for</code> vs <code>while</code>	知道次数用 <code>for</code> ，不知道次数、看条件用 <code>while</code>
<code>console.log()</code>	你和浏览器对话的窗口，调试第一步永远是打印看看